

DCSPitHIDC

Built for DCS World. Engineered to outperform.

A single-board USB controller that turns your switches, encoders, and warning lights into a fully integrated cockpit — configured entirely from a **Windows application**, with **no code, no compiler, and no development environment**. Wire the panel however suits you: the board works out what you connected, and where.

WHY IT'S DIFFERENT

Your configuration is data, not code. Nothing you do requires programming. Select a cockpit function, press the physical switch, and it is bound — the same interaction you already use to bind a control inside DCS. Turn an encoder and the board detects which pins it is wired to and configures its own decoder. The firmware is written once, by us; you never open a toolchain.

True composite USB device — HID + CDC on one USB-C cable. The board enumerates simultaneously as a game controller and a configuration port over a single connection. No adapters, no second cable, no driver install.

Bare-metal firmware on USBX standalone — no RTOS. A lean, deterministic core with no scheduler between your hardware and the sim. Native inputs and encoders are sampled every **1 ms** — the same rate USB Full-Speed polls HID, which is the physical limit of the interface.

The board tells you what is wired where — not the other way round. Press a control and watch the exact pin light up on screen, labelled with its function. A pin lit while you touch nothing is a short to ground. Encoders announce themselves when turned. Faults are found in seconds, on the bench, rather than in the air.

Pre-configuration is a convenience, not a requirement.

The F/A-18C Hornet database ships ready to use, and boards can leave the factory already configured for a specific panel. But **none of that is mandatory**. If your aircraft — or your simulator — is not on the list, the board still works: create your own entries, press your switches, and fly. It is a standard HID controller, so **any flight simulator sees it**.

AT A GLANCE

MCU	STM32H743VIT6 — Arm Cortex-M7, 400 MHz
Firmware	USBX standalone, bare-metal — no RTOS
USB	Single USB Type-C — composite HID + CDC ACM, no driver install
Inputs	61 native + 16 via I/O expander — 77 total, on 2.54 mm headers
Encoders	Up to 25 EC11 quadrature — auto-detected, hardware-timer decoded
Sampling	1 ms — native inputs and encoders (1000 Hz)
Outputs	DCS-BIOS driven — LEDs and warning lights, any free pin
Backlight	Firmware PWM dimmer — operates with the simulator closed

INPUTS

NATIVE PINS

61 usable GPIO across ports A-E on 2.54 mm headers, wired directly to switches and buttons. Every pin is silkscreened on the board and labelled identically in the app — what you read on the PCB is what you see on screen.

Sampled every 1 ms. Reports are sent to the host the moment a state changes, at the same 1 ms cadence USB Full-Speed polls HID. Fast toggles and bouncy contacts cannot fall between samples.

128-button HID report. The board appears as an ordinary game controller — no drivers, and any simulator that reads a joystick will read it.

I/O EXPANDER

+16 inputs via an MCP23017 module on the dedicated 4-pin I2C header. Scanned, activated and deactivated live from the app; the state is remembered on the board.

Inputs only, by design. I2C is too slow to decode quadrature reliably, and expander pins cannot drive LEDs. Encoders and outputs therefore live on native pins — where they belong.

Expander pins are refreshed on their own cadence so the slower I2C bus never delays the 1 ms native scan.

ENCODERS

AUTOMATIC DETECTION

Wire an EC11 to any two free pins. Open the encoder page and turn the knob: the board detects the pin pair from the quadrature signal itself, shows you which two pins it found, and configures the decoder on confirmation.

No squinting at the silkscreen, no pin dropdowns, no detent-ratio setting, no A/B swapping. If you wired it to the wrong pins, turn it again and it re-detects.

Up to **25 encoders**, each with its own clockwise and counter-clockwise button.

HARDWARE-TIMED DECODING

Quadrature sampled by a dedicated 1 ms hardware-timer interrupt — not lazy polling in a loop. Fast spins do not drop steps, and there is no double-stepping or jitter.

Verified, not assumed. The wiring check confirms each encoder by lighting one pin for clockwise and the other for counter-clockwise. Both directions proven, on the bench, before you fly.

Encoder pins are excluded from the button scan automatically — one pin, one job.

Backlight dimmer — the only thing on the board that needs nothing at all

Any encoder can be declared a **console backlight dimmer**. It drives a PWM pin directly in firmware, so your panel lights up **with the simulator closed, the app closed, and the PC idle** — plug the board into a phone charger and the knob still works. At the same time it keeps sending its rotation to the sim, so the in-game knob follows your physical one. It starts dark at power-up, exactly as a real console dimmer does, and the board reports its true brightness back to the app.

O U T P U T S

COCKPIT-DRIVEN LIGHTS

LED outputs driven by live DCS-BIOS data — APU Ready, Master Caution, gear and flap indicators, caution and advisory panels — reflecting the actual state of the aircraft in real time, over the same single cable.

Fully assignable. Any cockpit output to any free pin, chosen in the app. Address, mask and shift are handled for you, and remain editable if you need to point at something the database does not know.

WINDOWS CONFIGURATION APP

One-click connect over the COM port — no drivers to install.

Bind by pressing. Choose the function, press the switch. Press the same switch on another function and the binding moves across — exactly as DCS behaves.

Edit anything, at any time. Wrong function, wrong pin, wrong address — click Edit and change it in place. Nothing is ever locked in.

Free Function creates a custom entry for anything not in the database — any aircraft, any simulator.

D I A G N O S T I C S

SEE WHAT THE BOARD SEES

- **Live wiring check.** Press any control — the exact pin lights up on screen, labelled with the function assigned to it. Native and expander pins in one view.
- **Short-circuit detection.** A pin lit while you are touching nothing is shorted to ground. It is visible immediately, not after an evening of hunting.
- **Encoder verification.** Turn each knob both ways; the pins confirm green once both directions are proven.
- **Unconfigured encoders are flagged** — a flickering pin pair is detected and offered for setup.

PROVE IT BEFORE YOU FLY

- **One-click self-test.** A pass/fail report across six groups: button numbering, current config, encoders, function database, binding export, and device sync. Duplicate pins and binding collisions are caught before they reach the cockpit.
- **Real-time debug log.** Every byte to and from the board, timestamped, live on screen and written to file. When something is genuinely wrong, you read the cause rather than guess it.
- For panel builders and resellers, this turns support into evidence: **"send me the log."**

S I M I N T E G R A T I O N & S T O R A G E

DCS BINDINGS, WRITTEN FOR YOU

.diff.lua export. The app writes your DCS binding file — native switches, expander switches and encoders, each with the correct joystick button number. Drop it into DCS and fly. No hand-editing Lua.

F/A-18C Hornet database included: 125 cockpit outputs, 563 inputs and 57 encoder functions, ready to select. **Any other DCS module** is supported through DCS-BIOS.

Any other simulator sees a standard HID controller — MSFS, X-Plane, Falcon BMS and the rest read it as a joystick without any special support.

PERSISTENT STORAGE

8 MB external QSPI flash (Winbond W25Q64JV). Your configuration survives power-off and travels with the board.

Test before you commit. Push a configuration to the board's memory to try it live, then write it permanently once you are satisfied. **Load from Device** reads back whatever is on a board.

Ships ready to run. Boards can be fully configured before shipping — ideal for panel makers delivering plug-and-play products.

FROM BARE PANEL TO FLYING

01	02	03	04
Wire it	Press it	Test it	Fly it
Solder switches, encoders and LEDs to whatever pins route most conveniently. There is no prescribed pinout to design your panel around.	Choose a cockpit function, press the switch. Turn each encoder — it is detected and configured for you.	Run the wiring check and the self-test. Shorts, conflicts and duplicate pins surface here, not in the cockpit.	Save the configuration to the board, export the DCS binding file, and go. The board is a plain game controller — no drivers.

WHERE IT STANDS TODAY

The board is complete and the software is in active development. This is what is in your hands now, and what is coming next.

- **Buttons and switches you simply press — encoders and LED outputs are assigned deliberately.** An LED cannot report which pin it is on, and an encoder must be declared so the firmware decodes it cleanly rather than treating it as two noisy buttons. That declaration is what removes the jitter.
- **The I/O expander adds inputs only.** I2C is too slow to decode quadrature reliably, and expander pins cannot drive LEDs. Encoders and outputs therefore live on native pins, where they perform properly.
- **The curated database currently covers the F/A-18C Hornet** — 125 outputs, 563 inputs, 57 encoders. Every other DCS module already works through DCS-BIOS today; curated per-aircraft databases are being added module by module.
- **Cockpit-driven lights are DCS-BIOS based.** Inputs work in any simulator that reads a joystick; output support for other simulators is in development.
- **The configuration app is Windows.**

COMING IN FUTURE UPDATES

- **Curated databases for the remaining DCS modules** — added progressively, delivered as a free app update. The board itself needs nothing.
- **Output callbacks for other simulators**, so cockpit lights are not DCS-only.
- **Wiring-guide export** — a printable pin-by-pin diagram generated from your own configuration.

Every update is **software only** and free. The hardware you buy today runs everything above.

The short version

Wire it however you like. Press the switches. Turn the knobs. The board works out the rest, tells you if you made a mistake, writes your DCS bindings for you, and keeps the whole configuration in its own memory so it still works on a machine that has never seen the app. **No code. No compiler. No development environment.**